Sentiment Analysis of Movie Reviews: Can a Model Understand "Not"?

CSC 425 AI Term Project, Milestone 3: Working AI System & Preliminary Evaluation
Obaid Alnaqbi (solo project)

1. Introduction

This project determines whether a movie review is positive or negative and poses a single question: is the model able to detect that the word "not" changes the meaning of a review? The two sentences "The film was boring" and "The film was not boring" differ by one word but have opposite meanings.

My best TF-IDF baseline at Milestone 2 reached 88% validation accuracy, but got at most 3 of my 10 negation pairs fully correct, since it learnt the word "not" as a plain negative word. For this milestone, I experimented with three changes and measured each one:

- **Regularization:** tune the regularization of each model on the validation set;
- **Negation marking:** attach a negation marker to the words following "not", so that "boring" and "not_boring" become different features;
- **More data:** train on up to all 24,200 available training reviews, as opposed to 3,200.

Then I chose the final model with a rule fixed in advance, and tested it only once on the official 25,000-review IMDb test set.

2. Dataset and Problem Environment

The data is the IMDB Large Movie Review Dataset (Maas et al., 2011): 25,000 training and 25,000 test reviews, balanced, labeled by star rating (4/10 or lower negative, 7/10 or higher positive). The Milestone 2 split (3,200 training and 800 validation reviews) is used for development. The learning curve keeps the same 800 validation reviews while more training reviews are added from the other 24,200. The official test set was used only once, at the end. Preprocessing is unchanged (remove
 tags, lowercase, keep the stop words).

Additionally, three small test sets are hand-written. They are just for analysis, not training, and each was written before any model was run on it:

- **Negation pairs:** 10 sentence pairs where adding "not" reverses the opinion (from Milestone 2).
- **Mixed opinions (new):** 10 reviews that include both good and bad comments, labeled by the overall opinion, such as "It starts slow, but the last hour is brilliant." (positive).
- **Sarcasm (new):** 10 sarcastic reviews that use positive words to say something negative, e.g. "What a masterpiece, I only fell asleep twice." All 10 of them are negative, so a model that leans towards "negative" will do well on this set; I keep that in mind when reading the scores.

3. Methodology

Models

The models are the Milestone 2 pipelines: a TF-IDF vector per review, then Logistic Regression ($P(\text{positive}) = \text{sigmoid}(w \cdot x + b)$, where C is the regularization setting: larger C means weaker

regularization) or Multinomial Naive Bayes (alpha is the smoothing added to each count). Features are single words (ngram_range = (1, 1)) or words plus neighbouring word pairs ((1, 2)).

Negation marking

This is the usual method for handling negation in a bag-of-words model (Das & Chen, 2001; Pang et al., 2002). First, contractions are expanded ("didn't" to "did not"), because the default tokenizer splits "didn't" into "didn" and the negation is lost. Then every word that follows a negation word (not, no, never, nothing, without, ...) gets a not_ prefix until the next punctuation mark or a contrast word (but, however, although, though, yet):

"the film was not boring at all." becomes "the film was not not_boring not_at not_all ."

Now "boring" (negative) and "not_boring" are two separate features with separate weights, so in theory the model can learn that "not_boring" is positive. Crossing the two feature types with negation marking on or off gives four settings: words, words+pairs, words+neg, and words+pairs+neg.

4. Experimental Setup

1. **Tuning:** each classifier and feature setting was tuned with C in {0.1, 0.3, ..., 1000} or alpha in {0.01, ..., 1}, and the value with the highest validation F1 was kept.
2. **Negation comparison:** all 8 tuned models (2 classifiers x 4 feature settings) were evaluated on the validation set and all three challenge sets.
3. **Stability check:** 800 validation reviews is a small sample (a 95% interval for accuracy is about ±2 points), so I repeated the same Logistic Regression comparison on five other random 3,200/800 splits.
4. **Negation scope:** mark 1, 2 or 3 words following "not", or until punctuation.
5. **Learning curve:** 3,200, 6,400, 12,800 and 24,200 training reviews, same validation set.
6. **Final test:** choose the configuration with the highest validation F1 (ties within 0.005 go to the one with more negation pairs correct), train it on all 25,000 training reviews, and evaluate it once on the test set next to the Milestone 2 baseline.

Metrics: accuracy, precision, recall and F1 on real reviews; correct sentences and fully correct pairs on the challenge sets. scikit-learn 1.7, seed 42, about four minutes on a laptop CPU (python src/run_milestone3.py).

5. Preliminary Results

Tuning

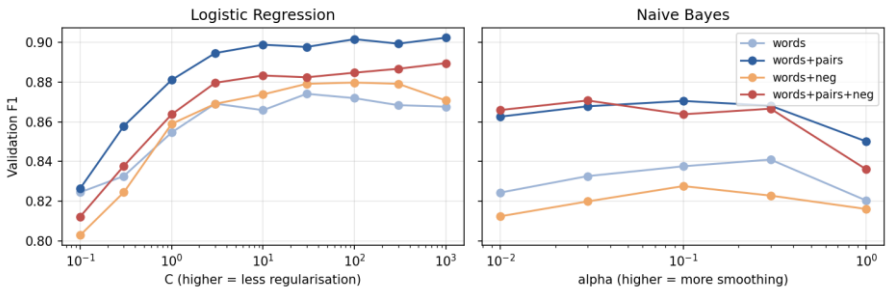


Figure 1. Validation F1 against the regularization setting (log scale).

Tuning improved all of the models. Logistic Regression with word pairs achieved 0.881 F1 with the default C = 1, and 0.902 at the best C. The curve is nearly flat from C = 10 to C = 1000 (0.898 to 0.902), so the exact value is not important as long as the regularization is not strong. Naive Bayes preferred less smoothing (alpha = 0.03 to 0.3) than the default alpha = 1. One side effect of a very large C is that the model becomes overconfident: it often predicts a probability of 0.99 or 1.00, even when it is wrong.

Negation marking

Model	Features	C / alpha	# features	Acc.	F1	Neg. pairs	Mixed	Sarcasm
Logistic Regression	words	30	17,291	0.875	0.874	3/10	8/10	5/10
Logistic Regression	words+pairs	1000	82,919	0.902	0.902	4/10	8/10	7/10
Logistic Regression	words+neg	100	20,207	0.880	0.880	5/10	7/10	6/10
Logistic Regression	words+pairs+neg	1000	86,648	0.891	0.889	6/10	8/10	7/10
Naive Bayes	words	0.3	17,291	0.849	0.841	2/10	9/10	9/10
Naive Bayes	words+pairs	0.1	82,919	0.875	0.870	3/10	9/10	9/10
Naive Bayes	words+neg	0.1	20,207	0.835	0.828	5/10	9/10	9/10
Naive Bayes	words+pairs+neg	0.03	86,648	0.874	0.871	3/10	9/10	8/10

Table 1. Tuned models on one split: 800 validation reviews and the three challenge sets.

On this split, negation marking increased the number of negation pairs correct (Logistic Regression: 3 to 5 with single words, 4 to 6 with word pairs), but lowered validation F1 slightly when used with word pairs (0.902 to 0.889). Since these are small differences on a small sample, I wanted to see if they hold up on five additional random splits (Figure 2).

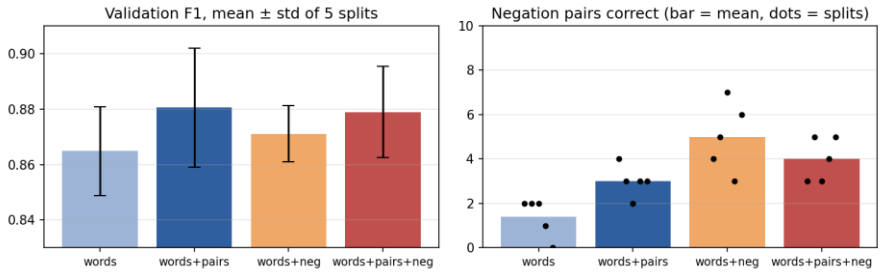


Figure 2. Logistic Regression on five other random 3,200/800 splits.

The stability check turns the picture upside down. In F1, the differences between the four settings (0.865 to 0.881) are smaller than the variation from one split to another (standard deviation 0.010 to 0.021), so on real reviews they are essentially tied, apart from single words being slightly worse. The negation result, however, is consistent: with single words, negation marking raised the number of negation pairs from 1.4 to 5.0 on average and won on all five splits (its worst split, 3 pairs, beats the best split without it, 2 pairs). The gain is smaller with word pairs (3.0 to 4.0). So the single-split result that word pairs alone are the best option was partly the luck of the draw.

Negation scope and more data

Marking only the next 1 or 2 words gave the best F1 (0.900 to 0.901), while marking until punctuation gave the most negation pairs (6 of 10). Again, the differences are within split-to-split noise, so I stuck with the simple until-punctuation rule.

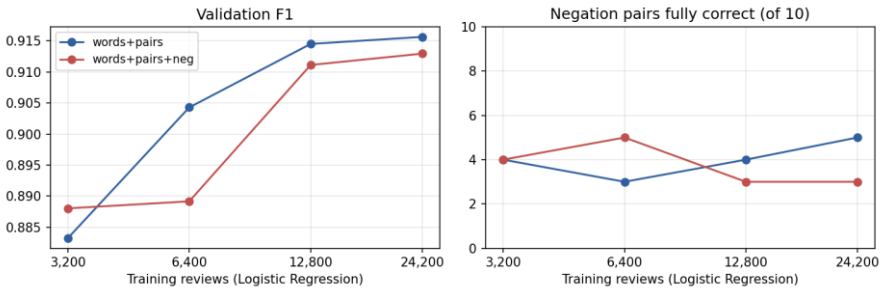


Figure 3. Logistic Regression trained on more data, evaluated on the same 800 validation reviews.

More data clearly helps on real reviews: F1 rises from 0.883 with 3,200 reviews to 0.916 with 24,200. However, it doesn't help on negation: the number of correct pairs stays between 3 and 5 at every size, with or without marking. This was the most surprising result, and it is explained in the next section.

The final model on the official test set

Model	Train reviews	Acc.	Prec.	Recall	F1	Neg. pairs	Mixed	Sarcasm
M2 baseline	3,200	0.856	0.852	0.862	0.857	3/10	8/10	4/10
Final	**25,000**	**0.905**	**0.906**	**0.904**	**0.905**	**4/10**	**9/10**	**7/10**

Table 2. Official IMDb test set (25,000 reviews), evaluated once. Both models are Logistic Regression with words + pairs; the final model uses C = 1000 and all training data.

By the fixed selection rule, the model with word pairs and no negation marking (highest single-split F1, 0.902) was selected as the final one. Trained on all 25,000 training reviews, it achieved a test accuracy of 90.5% and F1 of 0.905, while the Milestone 2 baseline achieved 85.6%. Its errors are balanced: 1,167 negative reviews are labeled positive, and 1,196 positive reviews are labeled negative. It also did better on mixed opinions (9/10) and sarcasm (7/10), but it still only manages 4/10 negation pairs.

6. Initial Discussion

Why more data does not teach negation

To understand the learning curve, I looked at how these phrases are used in real IMDb reviews, and what percentage of the reviews containing each phrase are positive, in all 25,000 training reviews:

Phrase	not bad	not great	not a great	not boring	not good	not funny	bad	good
Reviews	274	128	79	19	184	298	5,904	9,641
% positive	33%	52%	59%	53%	24%	7%	25%	50%

Table 3. Number of reviews that include each phrase in the 25,000 training reviews, and the percentage of those reviews that are positive.

This explains a lot. "not bad" is more common in negative reviews than in positive ones, probably in sentences such as "the acting was not bad, but the story was a mess". "not great" is split evenly, and "good" on its own does not indicate the rating. The model learns from review-level labels: it only learns that a phrase goes with a rating, not what the phrase means in its own sentence. My challenge set asks about sentence-level meaning, so even with perfect negation

features and all of the IMDb data, a model trained this way would still get "The sequel was not bad." wrong, because on IMDb "not bad" really does predict a bad review. Negation marking still helps when the negated word is used consistently ("not worth", "not recommend", "not funny"), which is why it improved the pair scores.

Other observations

- **Label noise and real sarcasm.** Several of the test reviews that the model got wrong with near-100% confidence look mislabeled or sarcastic. One labeled negative says "This is definitely one of the best Kung fu movies in the history of Cinema"; another says "Master P's acting skills make you actually believe he is Italian!", which is sarcasm. So some of the remaining 9.5% error is noise that the model cannot fix.
- **Overconfidence.** With C = 1000, the final model says "negative (1.00)" for "The film was not boring.". A confidence score that is always near 0 or 1 is misleading for a user. Since F1 is nearly flat from C = 10 upward, a smaller C would give more honest confidence without losing much accuracy.
- **Mixed opinions vs sarcasm.** Mixed opinions went well (9/10): words such as "brilliant", "wonderful" or "awful" tend to appear in the part of the review that reflects the overall opinion. Sarcasm is more difficult (7/10), and the sentences it gets wrong contain only positive words ("Truly the best film ever made, if you enjoy staring at a wall.").

What I learned: one validation split can mislead when the differences are small; accuracy on real reviews and understanding of negation are different skills, and more data helps with the first but not the second; and review-level ratings can't teach sentence-level meaning.

7. Remaining Work Plan

1. **Final system:** use words+pairs+neg (tied on F1 in the stability check, better on negation) and report its test result once, labeled as a second test-set evaluation.
2. **Adjust C:** check calibration and select a less extreme C.
3. **Domain extension:** use the movie model on product reviews, as advised in the Milestone 1 feedback.
4. **Demo:** a web page that displays the prediction, the confidence, and the words that pushed it in each direction.
5. **Deliverables:** final report (with an ethics section), README, presentation (with a "guess the prediction" segment), and the submission ZIP.

References

[1] Das, S., & Chen, M. (2001). Yahoo! for Amazon: Extracting market sentiment from stock message boards. Proceedings of the Asia Pacific Finance Association Annual Conference.

[2] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of ACL 2011, 142-150.

[3] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP 2002, 79-86.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[5] Stanford AI Lab. Large Movie Review Dataset. https://ai.stanford.edu/~amaas/data/sentiment/ (Hugging Face mirror: https://huggingface.co/datasets/stanfordnlp/imdb).